

CUADERNO JUPYTER UNIFICADO: Práctica 1 – Metaheurísticas (Curso 2025/2026)

Simulated Annealing (SA)

1. Introducción

El objetivo de esta práctica es segmentar una serie temporal en k segmentos, ajustando en cada uno de ellos una regresión lineal independiente. La calidad de una segmentación se evalúa mediante el **RMSE medio** de todos los segmentos, que actúa como función objetivo a minimizar.

En este cuaderno se implementa el algoritmo **Simulated Annealing (SA)**, una metaheurística inspirada en el proceso físico de recocido de metales. SA permite escapar de óptimos locales aceptando soluciones peores con una probabilidad controlada por la temperatura.

Simulated Annealing sigue una estrategia más flexible que Hill Climbing:

- Parte de una solución inicial aleatoria.
- Genera soluciones vecinas modificando los puntos de corte.
- Acepta soluciones peores con una probabilidad dependiente de la temperatura.
- Reduce la temperatura según una función de enfriamiento.
- Busca equilibrar exploración (temperaturas altas) y explotación (temperaturas bajas).

2. Importación de librerías

En esta sección se importan las librerías necesarias para:

- Cálculo numérico (`numpy`)
- Visualización (`matplotlib`)
- Ajuste de regresiones lineales (`sklearn`)
- Gestión de datos (`pandas`)
- Medición del error (`mean_squared_error`)
- Generación de números aleatorios
- Funciones matemáticas (`math`)
- Medición de tiempos

Estas herramientas permiten implementar y evaluar el algoritmo Simulated Annealing.

```
In [1]: from enum import Flag
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import random
import math
import time
from tabulate import tabulate
```

3. Carga de series temporales

Se define una función auxiliar para leer los archivos de datos que contienen las series temporales TS1-TS4.

Cada archivo contiene una lista de valores numéricos entre corchetes, por lo que es necesario limpiarlos antes de convertirlos a `float`.

Posteriormente, se cargan las cuatro series y se asocian con sus valores de `k` correspondientes:

- TS1 $\rightarrow k = 9$
- TS2 $\rightarrow k = 10$
- TS3 $\rightarrow k = 20$
- TS4 $\rightarrow k = 50$

Estas series serán utilizadas en los experimentos posteriores.

```
In [2]: def read_serie(path):
        with open(path, "r") as f:
            contenido = f.read().replace("[", "").replace("]", "").split()
            return [float(x) for x in contenido]

files = ["../datos/TS1", "../datos/TS2", "../datos/TS3", "../datos/TS4"]
series = [read_serie(f) for f in files]
k_values = [9, 10, 20, 50]

print("Series cargadas correctamente")
```

Series cargadas correctamente

4. Regresión lineal por segmentos

Para evaluar la calidad de una segmentación, SA necesita calcular el RMSE medio de los segmentos.

Para ello se implementan funciones que permiten ajustar una regresión lineal a cada segmento.

estimate_segment_coef(x, y)

Ajusta una recta mediante `scikit-learn`.

estimate_coef(x, y)

Calcula los coeficientes de forma manual usando fórmulas estadísticas.

estimate_all_coef(temp, points)

Calcula los coeficientes de todos los segmentos definidos por los puntos de corte.

estimate_all_points(coef, points, temp_size)

Reconstruye la serie estimada completa a partir de los coeficientes de cada segmento.

Estas funciones permiten evaluar la calidad de una segmentación.

```
In [3]: def estimate_segment_coef(x, y):
        x = np.array(x).reshape(-1, 1)
        y = np.array(y)
        if len(x) < 2:
            return (0.0, 0.0)
        model = LinearRegression().fit(x, y)
        return (model.coef_[0], model.intercept_)

def estimate_coef(x, y):
    x = np.array(x)
    y = np.array(y)
    n = np.size(x)

    if n == 0:
        return (0.0, 0.0)

    m_x = np.mean(x)
    m_y = np.mean(y)

    SS_xy = np.sum(y * x) - n * m_y * m_x
    SS_xx = np.sum(x * x) - n * m_x * m_x

    if abs(SS_xx) < 1e-8:
        b_0 = 0.0
        b_1 = m_y
    else:
        b_0 = SS_xy / SS_xx
        b_1 = m_y - b_0 * m_x

    return (float(b_0), float(b_1))

def estimate_all_coef(temp, points):
    coef = []
    start = 0

    pts = points.copy()
    pts.append(len(temp))
```

```

    for point in pts:
        b_0, b_1 = estimate_coef(range(start, point), temp[start:point])
        coef.append((b_0, b_1))
        start = point

    return coef

def estimate_point(coef, i):
    return i * coef[0] + coef[1]

def estimate_all_points(coef, points, temp_size):
    estimated = []
    start = 0

    pts = points.copy()
    pts.append(temp_size)

    for pos, point in enumerate(pts):
        for i in range(start, point):
            estimated.append(estimate_point(coef[pos], i))
        start = point

    return estimated

```

5. Generación de puntos de corte aleatorios

La función `randomPoints` genera una solución inicial aleatoria para Simulated Annealing.

Los puntos de corte se generan sin repetición y ordenados, garantizando que los segmentos sean válidos.

```

In [4]: def randomPoints(longitud_serie, n_cortes):
        b = []
        while n_cortes > 0:
            r = random.randint(1, longitud_serie - 1)
            if r not in b:
                b.append(r)
                n_cortes -= 1
        b.sort()
        return b

```

6. Cálculo del RMSE por segmentos

Para evaluar la calidad de una segmentación se utiliza el **RMSE medio** de todos los segmentos.

RMSE(y_real, y_pred)

Calcula el error cuadrático medio entre valores reales y estimados.

mean_rmse(serie, points)

Calcula el RMSE medio de todos los segmentos definidos por los puntos de corte.

Este valor es la función objetivo que Simulated Annealing intenta minimizar.

```
In [5]: def RMSE(y_real, y_pred):  
        mse = mean_squared_error(y_real, y_pred)  
        return np.sqrt(mse)  
  
def mean_rmse(serie, points):  
    rmse_acc = 0.0  
    start = 0  
    coef = estimate_all_coef(serie, points.copy())  
    estimated = estimate_all_points(coef, points.copy(), len(serie))  
    pts = points.copy()  
    pts.append(len(serie))  
    for point in pts:  
        rmse_acc += RMSE(serie[start:point], estimated[start:point])  
        start = point  
    return rmse_acc / len(pts)
```

6. Implementación del algoritmo Simulated Annealing (SA)

Simulated Annealing es una metaheurística inspirada en el proceso físico de recocido de metales.

El algoritmo parte de una solución inicial aleatoria y explora su vecindario aceptando:

- Soluciones mejores (siempre)
- Soluciones peores (con cierta probabilidad)

La probabilidad de aceptar soluciones peores depende de la **temperatura**, que se va reduciendo progresivamente según una función de enfriamiento.

Este mecanismo permite a SA escapar de óptimos locales y explorar regiones del espacio de soluciones que otros métodos, como Hill Climbing, no pueden alcanzar.

6.1 Funciones de enfriamiento

La temperatura controla la probabilidad de aceptar soluciones peores.

A medida que avanza el algoritmo, la temperatura debe disminuir para favorecer la explotación.

En este cuaderno se implementan tres funciones de enfriamiento:

Enfriamiento lineal

$$T_{i+1} = T_0 - i \cdot p$$

Enfriamiento exponencial

$$T_{i+1} = T_0 \cdot p^i$$

Enfriamiento logarítmico

$$T_{i+1} = \frac{T_0}{1 + \log(i)}$$

Cada una de estas funciones afecta de forma distinta al equilibrio entre exploración y explotación.

```
In [6]: enfriamiento_lineal = lambda T, T0, i, p: T0 - (i * p)
enfriamiento_exponencial = lambda T, T0, i, p: T0 * p**i
enfriamiento_logaritmico = lambda T, T0, i, p: T0 / (1 + math.log(i))
```

6.2 Generación de vecinos

Para explorar el espacio de soluciones, SA necesita generar soluciones vecinas a la actual.

Un vecino se obtiene modificando uno de los puntos de corte en $\pm 1\%$ del tamaño de la serie, respetando:

- Que ningún punto sea 0
- Que ningún punto sea igual a la longitud total
- Que los puntos estén estrictamente ordenados
- Que no existan duplicados
- Que no aparezcan segmentos vacíos

Este proceso garantiza que todos los vecinos representan segmentaciones válidas.

```
In [7]: def random_neighbour(points, n):
    neighbour = []
    flag = True

    while flag:
        neighbour = points.copy()
        neighbour[random.randint(0, len(neighbour) - 1)] += random.choice
            (-1, 1)
        ) * int(n / 50)

        if neighbour[0] <= 0:
            continue
        if neighbour[-1] >= n:
            continue
        if sorted(neighbour) != neighbour:
            continue
        if len(set(neighbour)) != len(neighbour):
            continue
```

```

        flag = False

    return neighbour

```

6.3 Algoritmo Simulated Annealing

El algoritmo completo sigue los siguientes pasos:

1. Generar una solución inicial aleatoria.
2. Calcular su RMSE.
3. Mientras la temperatura sea mayor que el umbral final:
 - Generar un vecino aleatorio.
 - Calcular su RMSE.
 - Si mejora, aceptar la solución.
 - Si empeora, aceptarla con probabilidad:

$$P = e^{-\Delta/T}$$

- Actualizar la temperatura según la función de enfriamiento.
- 4. Devolver la mejor solución encontrada.

El algoritmo devuelve:

- La mejor segmentación encontrada
- Su RMSE
- El tiempo total de ejecución

```

In [8]: def simulated_annealing(T0, funcion_enfriamiento, p, L, Tf, serie, k):
        startTime = time.time()

        T = T0
        sol = randomPoints(len(serie), k)
        rmse_sol = mean_rmse(serie, sol)
        best = sol.copy()
        rmse_best = rmse_sol
        iter = 1

        while T >= Tf:
            for _ in range(L):
                vecino = random_neighbour(sol, len(serie))
                rmse_vecino = mean_rmse(serie, vecino)

                delta = rmse_vecino - rmse_sol

                if delta < 0 or random.uniform(0, 1) < math.exp(-delta / max(
                    sol = vecino
                    rmse_sol = rmse_vecino

                if rmse_sol < rmse_best:
                    best = sol.copy()
                    rmse_best = rmse_sol

            T = funcion_enfriamiento(T, T0, iter, p)
            iter += 1

```

```
endTime = time.time()
return {"rmse": rmse_best, "points": best}, endTime - startTime
```

7. Ejecución del experimento completo

Para evaluar el rendimiento del algoritmo, se ejecuta Simulated Annealing:

- 20 veces por cada serie temporal
- Probando distintos valores de los parámetros:
 - Función de enfriamiento
 - Parámetro p
 - Número de iteraciones internas L
 - Temperaturas inicial y final

En cada repetición se almacena:

- El RMSE final
- Los puntos de corte óptimos
- El tiempo de ejecución

Estos datos permiten analizar cómo afectan los parámetros al rendimiento del algoritmo.

```
In [9]: def ejecutar_experimento(
        series,
        k_values,
        nombre,
        funcion_enfriamiento,
        p=None,
        T0=100,
        Tf=0.1,
    ):
    print(f"\n=== Ejecutando SA con enfriamiento {nombre} ===")

    mean_bests = []
    mean_times = []

    for i in range(20):
        series_bests = []
        series_times = []
        for TS in range(len(series)):

            current_serie = series[TS]
            current_k = k_values[TS]

            best, time_exec = simulated_annealing(
                T0=T0,
                funcion_enfriamiento=funcion_enfriamiento,
                L=50,
                Tf=Tf,
                serie=current_serie,
                k=current_k,
                p=p)
```

```
        series_bests.append(best)
        series_times.append(time_exec)
    print(f"Repetición {i + 1}/{20} completada")

    mean_bests.append(series_bests)
    mean_times.append(series_times)

print("=== Experimento finalizado ===")
return mean_bests, mean_times

def ejecutar_experimento_p(
    series,
    k_values,
    nombre,
    funcion_enfriamiento,
    pi=1,
    pf=200,
    step=20,
    T0=100,
    Tf=0.1,
    L=20,
):
    print(f"\n=== Ejecutando SA con enfriamiento {nombre} ===")

    mean_bests = []
    mean_times = []
    p=[]
    for i in range(20):
        series_bests = []
        series_times = []
        for TS in range(len(series)):

            current_serie = series[TS]
            current_k = k_values[TS]

            bests=[]
            times=[]
            for p_val in range(pi,pf,step):
                best, time_exec = simulated_annealing(
                    T0=T0,
                    funcion_enfriamiento=funcion_enfriamiento,
                    L=L,
                    Tf=Tf,
                    serie=current_serie,
                    k=current_k,
                    p=p_val)
                bests.append(best)
                times.append(time_exec)
                if TS==0 and i==0:
                    p.append(p_val)

            series_bests.append(bests)
            series_times.append(times)
        print(f"Repetición {i + 1}/{20} completada")

    mean_bests.append(series_bests)
    mean_times.append(series_times)
```

```

print("=== Experimento finalizado ===")
return mean_bests, mean_times, p

def ejecutar_experimento_L(
    series,
    k_values,
    nombre,
    funcion_enfriamiento,
    Li=10,
    Lf=100,
    p=200,
    step=20,
    T0=100,
    Tf=0.1,
    L=20,
):
    print(f"\n=== Ejecutando SA con enfriamiento {nombre} ===")

    mean_bests = []
    mean_times = []
    L=[]
    for i in range(20):
        series_bests = []
        series_times = []
        for TS in range(len(series)):

            current_serie = series[TS]
            current_k = k_values[TS]

            bests=[]
            times=[]
            for L_val in range(Li,Lf,step):
                best, time_exec = simulated_annealing(
                    T0=T0,
                    funcion_enfriamiento=funcion_enfriamiento,
                    L=L_val,
                    Tf=Tf,
                    serie=current_serie,
                    k=current_k,
                    p=p)
                bests.append(best)
                times.append(time_exec)
                if TS==0 and i==0:
                    L.append(L_val)

            series_bests.append(bests)
            series_times.append(times)
            print(f"Repetición {i + 1}/{20} completada")

            mean_bests.append(series_bests)
            mean_times.append(series_times)

    print("=== Experimento finalizado ===")
    return mean_bests, mean_times, L

```

7.1 Funciones auxiliares para experimentos

Se implementan funciones que permiten:

- Ejecutar SA con diferentes funciones de enfriamiento
- Probar distintos valores del parámetro p
- Evaluar el efecto del número de iteraciones internas L
- Analizar el impacto de las temperaturas inicial y final

Estas funciones automatizan la experimentación y permiten generar tablas comparativas.

```
In [10]: def create_Table_Lambda(bests_lin, times_lin, bests_exp, times_exp, bests_log):
    rows=[]
    bests=[]
    bests.append(bests_lin)
    bests.append(bests_exp)
    bests.append(bests_log)

    times=[]
    times.append(times_lin)
    times.append(times_exp)
    times.append(times_log)

    names=["Lineal", "Exponencial", "Logaritmico"]

    for N_val in range(len(names)):
        for TS in range(len(bests[0][0])):
            rmse = [bests[N_val][rep][TS]["rmse"] for rep in range(len(bests))]
            time = [times[N_val][rep][TS] for rep in range(len(bests))]
            rows.append({
                "Enfriamiento": names[N_val],
                "Serie": f"Serie {TS + 1}",
                "RMSE Medio": np.mean(rmse),
                "RMSE Desv": np.std(rmse),
                "Tiempo Medio (s)": np.mean(time)
            })

    tabla = pd.DataFrame(rows)
    print(tabulate(tabla, headers="keys", tablefmt="fancy_grid", showindex=True))

def create_Table_p(bests, times, p):
    rows=[]
    for p_val in range(len(p)):
        for TS in range(len(bests[0])):
            rmse = [bests[rep][TS][p_val]["rmse"] for rep in range(len(bests))]
            time = [times[rep][TS][p_val] for rep in range(len(bests))]
            rows.append({
                "p": p[p_val],
                "Serie": f"Serie {TS + 1}",
                "RMSE Medio": np.mean(rmse),
                "RMSE Desv": np.std(rmse),
                "Tiempo Medio (s)": np.mean(time)
            })
```

```

        tabla = pd.DataFrame(rows)
        print(tabulate(tabla, headers="keys", tablefmt="fancy_grid", showindex=True))

def create_Table_L(bests, times, L):
    rows=[]
    for L_val in range(len(L)):
        for TS in range(len(bests[0])):
            rmse = [bests[rep][TS][L_val]["rmse"] for rep in range(len(bests))]
            time = [times[rep][TS][L_val] for rep in range(len(bests))]
            rows.append({
                "L": L[L_val],
                "Serie": f"Serie {TS + 1}",
                "RMSE Medio": np.mean(rmse),
                "RMSE Desv": np.std(rmse),
                "Tiempo Medio (s)": np.mean(time)
            })

    tabla = pd.DataFrame(rows)
    print(tabulate(tabla, headers="keys", tablefmt="fancy_grid", showindex=True))

def create_Table_T(bests, times, T0,Tf):
    rows=[]
    for TS in range(len(bests[0])):
        rmse = [bests[rep][TS]["rmse"] for rep in range(len(bests))]
        time = [times[rep][TS] for rep in range(len(bests))]
        rows.append({
            "T0": T0,
            "Tf": Tf,
            "Serie": f"Serie {TS + 1}",
            "RMSE Medio": np.mean(rmse),
            "RMSE Desv": np.std(rmse),
            "Tiempo Medio (s)": np.mean(time)
        })
    tabla = pd.DataFrame(rows)
    print(tabulate(tabla, headers="keys", tablefmt="fancy_grid", showindex=True))

```

```

In [18]: def SA_Lambda(series, k_values):
    bests_exp, times_exp = ejecutar_experimento(
        series, k_values, "exponencial", enfriamiento_exponencial, p=0.5, Tf=20
    )

    bests_log, times_log = ejecutar_experimento(
        series, k_values, "logarítmico", enfriamiento_logaritmico, Tf=20
    )

    bests_lin, times_lin = ejecutar_experimento(
        series, k_values, "lineal", enfriamiento_lineal, p=10, Tf=20
    )

    create_Table_Lambda(bests_lin, times_lin,bests_exp, times_exp,bests_log, times_log)

def SA_p(series, k_values):
    pi = int(input("P inicial: "))
    pf = int(input("P final: "))
    step = int(input("Incremento: "))

```

```

bests, times, p = ejecutar_experimento_p(
series, k_values, "p", enfriamiento_lineal, pf=pf, pi=pi, step=step)
create_Table_p(bests, times, p)

def SA_L(series, k_values):
    Li = int(input("L inicial: "))
    Lf = int(input("L final: "))
    step = int(input("Incremento: "))

    bests, times, L = ejecutar_experimento_L(
series, k_values, "L", enfriamiento_lineal, Lf=Lf, Li=Li, step=step)
create_Table_L(bests, times, L)

def SA_T(series, k_values):
    T0 = float(input("T inicial: "))
    Tf = float(input("T final: "))
    p = float(input("Valor de p (enfriamiento lineal): "))

    bests, times = ejecutar_experimento(
series, k_values, "T", enfriamiento_lineal, p=p, Tf=Tf, T0=T0)

    create_Table_T(bests, times, T0, Tf)

```

8. Ejecución secuencial de todas las configuraciones de Simulated Annealing

En este apartado se ejecutan de forma automática las distintas variantes del algoritmo Simulated Annealing (SA), probando diferentes configuraciones de sus parámetros principales.

El objetivo es analizar cómo afectan estos parámetros al rendimiento del algoritmo en términos de RMSE, estabilidad y tiempo de cómputo.

Las opciones ejecutadas son:

1. Cambiar la función de enfriamiento λ

- Se comparan los esquemas lineal, exponencial y logarítmico.

2. Cambiar el número de iteraciones internas L

- Se evalúa cómo influye el número de vecinos explorados por nivel de temperatura.

3. Cambiar las temperaturas inicial y final (T_0 / T_f)

- Se estudia el impacto del rango térmico en la exploración del espacio de soluciones.

4. Cambiar el parámetro p del enfriamiento lineal

- Se analiza cómo la velocidad de enfriamiento afecta a la aceptación de soluciones peores.

Cada opción se ejecuta 20 veces para cada serie temporal, registrando:

- El **RMSE final**
- La **desviación típica**
- El **tiempo medio de ejecución**

Estos resultados permiten comparar de forma sistemática el comportamiento de SA bajo diferentes configuraciones y facilitan su posterior comparación con Random Search y Hill Climbing.

```
In [21]: # Lista de opciones que quieres ejecutar una debajo de otra
opciones = ["1", "2", "3", "4"]

print("=== EJECUCIÓN SECUENCIAL DE TODAS LAS OPCIONES DE SA ===")

for opcion in opciones:
    print(f"\n--- Ejecutando opción {opcion} ---")

    if opcion == "1":
        print(">> Cambiar lambda")
        SA_Lambda(series, k_values)

    elif opcion == "2":
        print(">> Cambiar número de iteraciones L")
        SA_L(series, k_values)

    elif opcion == "3":
        print(">> Cambiar temperatura T0 / Tf")
        SA_T(series, k_values)

    elif opcion == "4":
        print(">> Cambiar p en el enfriamiento lineal")
        SA_p(series, k_values)

print("\n=== FIN DE EJECUCIÓN ===")
```

=== EJECUCIÓN SECUENCIAL DE TODAS LAS OPCIONES DE SA ===

--- Ejecutando opción 1 ---
>> Cambiar lambda

=== Ejecutando SA con enfriamiento exponencial ===

Repetición 1/20 completada
Repetición 2/20 completada
Repetición 3/20 completada
Repetición 4/20 completada
Repetición 5/20 completada
Repetición 6/20 completada
Repetición 7/20 completada
Repetición 8/20 completada
Repetición 9/20 completada
Repetición 10/20 completada
Repetición 11/20 completada
Repetición 12/20 completada
Repetición 13/20 completada
Repetición 14/20 completada
Repetición 15/20 completada
Repetición 16/20 completada
Repetición 17/20 completada
Repetición 18/20 completada
Repetición 19/20 completada
Repetición 20/20 completada
=== Experimento finalizado ===

=== Ejecutando SA con enfriamiento logarítmico ===

Repetición 1/20 completada
Repetición 2/20 completada
Repetición 3/20 completada
Repetición 4/20 completada
Repetición 5/20 completada
Repetición 6/20 completada
Repetición 7/20 completada
Repetición 8/20 completada
Repetición 9/20 completada
Repetición 10/20 completada
Repetición 11/20 completada
Repetición 12/20 completada
Repetición 13/20 completada
Repetición 14/20 completada
Repetición 15/20 completada
Repetición 16/20 completada
Repetición 17/20 completada
Repetición 18/20 completada
Repetición 19/20 completada
Repetición 20/20 completada
=== Experimento finalizado ===

=== Ejecutando SA con enfriamiento lineal ===

Repetición 1/20 completada
Repetición 2/20 completada
Repetición 3/20 completada
Repetición 4/20 completada
Repetición 5/20 completada
Repetición 6/20 completada
Repetición 7/20 completada
Repetición 8/20 completada

Repetición 9/20 completada
Repetición 10/20 completada
Repetición 11/20 completada
Repetición 12/20 completada
Repetición 13/20 completada
Repetición 14/20 completada
Repetición 15/20 completada
Repetición 16/20 completada
Repetición 17/20 completada
Repetición 18/20 completada
Repetición 19/20 completada
Repetición 20/20 completada
=== Experimento finalizado ===

Enfriamiento (s)	Serie	RMSE Medio	RMSE Desv	Tiempo Medio
Lineal 4	Serie 1	0.633789	0.0635771	0.3621
Lineal 34	Serie 2	0.906794	0.0749648	0.4129
Lineal 08	Serie 3	0.778601	0.0362407	0.7691
Lineal 7	Serie 4	1.15664	0.0528304	1.7714
Exponencial 74	Serie 1	0.716158	0.0777204	0.3132
Exponencial 89	Serie 2	0.925415	0.111859	0.3526
Exponencial 86	Serie 3	0.795751	0.0531464	0.6780
Exponencial	Serie 4	1.17162	0.0751216	1.5336
Logaritmico 2	Serie 1	0.54334	0.0505055	2.5466
Logaritmico 7	Serie 2	0.791268	0.0556977	2.8961
Logaritmico	Serie 3	0.754191	0.0360031	4.6820

```
--- Ejecutando opción 2 ---
>> Cambiar número de iteraciones L
L inicial: 10
L final: 100
Incremento: 10
```

=== Ejecutando SA con enfriamiento L ===

Repetición 1/20 completada
Repetición 2/20 completada
Repetición 3/20 completada
Repetición 4/20 completada
Repetición 5/20 completada
Repetición 6/20 completada
Repetición 7/20 completada
Repetición 8/20 completada
Repetición 9/20 completada
Repetición 10/20 completada
Repetición 11/20 completada
Repetición 12/20 completada
Repetición 13/20 completada
Repetición 14/20 completada
Repetición 15/20 completada
Repetición 16/20 completada
Repetición 17/20 completada
Repetición 18/20 completada
Repetición 19/20 completada
Repetición 20/20 completada
=== Experimento finalizado ===

L	Serie	RMSE Medio	RMSE Desv	Tiempo Medio (s)
10	Serie 1	0.794465	0.126334	0.00933006
10	Serie 2	1.08163	0.111373	0.0102792
10	Serie 3	0.852412	0.0477942	0.0190042
10	Serie 4	1.24654	0.0695412	0.0446485
20	Serie 1	0.767158	0.10494	0.0170521
20	Serie 2	0.975562	0.0997322	0.0195637
20	Serie 3	0.851357	0.0431618	0.0364151
20	Serie 4	1.2151	0.070386	0.0838633
30	Serie 1	0.761446	0.0834318	0.0249827
30	Serie 2	1.00815	0.083159	0.0285946
30	Serie 3	0.837703	0.0425284	0.0541902
30	Serie 4	1.26658	0.0720074	0.123198
40	Serie 1	0.760292	0.0698189	0.0330412
40	Serie 2	1.00371	0.0771743	0.0385719
40	Serie 3	0.844089	0.0373045	0.0702436
40	Serie 4	1.20108	0.0653929	0.162408
50	Serie 1	0.713688	0.0947507	0.0412439
50	Serie 2	0.973921	0.0856351	0.046198

50	Serie 3	0.811769	0.0513344	0.0889162
50	Serie 4	1.17446	0.0614217	0.204519
60	Serie 1	0.731718	0.0885989	0.0495323
60	Serie 2	0.96939	0.0669841	0.0556087
60	Serie 3	0.809348	0.055196	0.104291
60	Serie 4	1.20337	0.0693267	0.242835
70	Serie 1	0.728004	0.0934232	0.0597999
70	Serie 2	0.957203	0.114183	0.0662092
70	Serie 3	0.82404	0.0406997	0.124293
70	Serie 4	1.19041	0.0695072	0.284659
80	Serie 1	0.756912	0.0748712	0.0690868
80	Serie 2	0.935048	0.138838	0.072794
80	Serie 3	0.815452	0.0421556	0.140488
80	Serie 4	1.2002	0.0670736	0.324996
90	Serie 1	0.733319	0.0878008	0.0763202
90	Serie 2	0.950895	0.0830737	0.0828202
90	Serie 3	0.798745	0.0595361	0.157028
90	Serie 4	1.20547	0.0500998	0.363946

--- Ejecutando opción 3 ---

>> Cambiar temperatura T0 / Tf

T inicial: 100

T final: 20

Valor de p (enfriamiento lineal): 10

=== Ejecutando SA con enfriamiento T ===

Repetición 1/20 completada
Repetición 2/20 completada
Repetición 3/20 completada
Repetición 4/20 completada
Repetición 5/20 completada
Repetición 6/20 completada
Repetición 7/20 completada
Repetición 8/20 completada
Repetición 9/20 completada
Repetición 10/20 completada
Repetición 11/20 completada
Repetición 12/20 completada
Repetición 13/20 completada
Repetición 14/20 completada
Repetición 15/20 completada
Repetición 16/20 completada
Repetición 17/20 completada
Repetición 18/20 completada
Repetición 19/20 completada
Repetición 20/20 completada
=== Experimento finalizado ===

T0	Tf	Serie	RMSE Medio	RMSE Desv	Tiempo Medio (s)
100	20	Serie 1	0.66329	0.066337	0.368658
100	20	Serie 2	0.889948	0.0691357	0.420119
100	20	Serie 3	0.791899	0.0334624	0.791256
100	20	Serie 4	1.16198	0.0532518	1.77468

--- Ejecutando opción 4 ---

>> Cambiar p en el enfriamiento lineal

P inicial: 1

P final: 50

Incremento: 5

=== Ejecutando SA con enfriamiento p ===

Repetición 1/20 completada
Repetición 2/20 completada
Repetición 3/20 completada
Repetición 4/20 completada
Repetición 5/20 completada
Repetición 6/20 completada
Repetición 7/20 completada
Repetición 8/20 completada
Repetición 9/20 completada
Repetición 10/20 completada
Repetición 11/20 completada
Repetición 12/20 completada
Repetición 13/20 completada
Repetición 14/20 completada
Repetición 15/20 completada
Repetición 16/20 completada
Repetición 17/20 completada
Repetición 18/20 completada
Repetición 19/20 completada
Repetición 20/20 completada
=== Experimento finalizado ===

p	Serie	RMSE Medio	RMSE Desv	Tiempo Medio (s)
1	Serie 1	0.56068	0.0437091	1.60877
1	Serie 2	0.800765	0.0802061	1.78785
1	Serie 3	0.75676	0.0424392	3.41474
1	Serie 4	1.10149	0.0656073	7.86164
6	Serie 1	0.656016	0.0627031	0.268786
6	Serie 2	0.887943	0.0810344	0.306528
6	Serie 3	0.78638	0.0548397	0.576763
6	Serie 4	1.14476	0.049799	1.34238
11	Serie 1	0.66987	0.0843659	0.160711
11	Serie 2	0.928139	0.0820787	0.180386
11	Serie 3	0.778274	0.0394836	0.340162
11	Serie 4	1.16133	0.0759081	0.789888
16	Serie 1	0.680686	0.0837923	0.10951
16	Serie 2	0.954709	0.101203	0.125209
16	Serie 3	0.815923	0.0366021	0.240757
16	Serie 4	1.18648	0.0638126	0.559021
21	Serie 1	0.70046	0.0837805	0.0796959
21	Serie 2	0.964094	0.0747056	0.0898803

21	Serie 3	0.794584	0.0360598	0.171147
21	Serie 4	1.17412	0.0503442	0.397063
26	Serie 1	0.723794	0.0759729	0.0634378
26	Serie 2	0.979422	0.0767963	0.0720703
26	Serie 3	0.821163	0.0467831	0.137121
26	Serie 4	1.19219	0.0479009	0.317103
31	Serie 1	0.740818	0.0816728	0.0629457
31	Serie 2	1.00866	0.0722084	0.0718716
31	Serie 3	0.804106	0.0464485	0.137016
31	Serie 4	1.1817	0.0544931	0.320997
36	Serie 1	0.7507	0.0812636	0.0474484
36	Serie 2	0.985121	0.0900427	0.0541951
36	Serie 3	0.823132	0.0397955	0.103319
36	Serie 4	1.20766	0.0650214	0.238827
41	Serie 1	0.735852	0.0741372	0.0480668
41	Serie 2	0.959208	0.11474	0.054524
41	Serie 3	0.843944	0.0384964	0.103435
41	Serie 4	1.21506	0.0646093	0.243066
46	Serie 1	0.703736	0.0959426	0.0480307
46	Serie 2	1.02266	0.0720048	0.0541905
46	Serie 3	0.822197	0.0373097	0.104101
46	Serie 4	1.20099	0.0480895	0.242901

=== FIN DE EJECUCIÓN ===